ICLOS FOR, WHILE Y DO-WHILE EN PROGRAMACIÓN

Introducción

En programación, los ciclos o estructuras repetitivas permiten ejecutar un bloque de instrucciones varias veces sin necesidad de escribir el mismo código repetidamente. Estas estructuras son fundamentales para automatizar tareas, recorrer arreglos, realizar cálculos y procesar grandes cantidades de información.

Entre los ciclos más conocidos se encuentran `for`, `while` y `do-while`. Cada uno tiene características particulares que permiten elegir la mejor opción según el problema que se desea resolver.

1. ¿Qué son los ciclos en programación?

Los ciclos son estructuras de control que permiten repetir una o varias instrucciones mientras se cumpla una condición o durante una cantidad determinada de repeticiones.

Características principales

- Permiten repetir instrucciones automáticamente.
- Reducen la repetición innecesaria de código.
- Facilitan el procesamiento de datos.
- Pueden utilizar condiciones para controlar su ejecución.
- Se utilizan para recorrer arreglos, contar elementos y realizar operaciones matemáticas.

2. Ciclo FOR

Definición

El ciclo `for` se utiliza para recorrer una secuencia de elementos o repetir instrucciones una cantidad determinada de veces. Es especialmente útil cuando se conoce el conjunto de datos que se desea recorrer o el número de iteraciones.

Sintaxis en Python

```
for variable in secuencia:
    instrucciones
```

La variable toma el valor de cada elemento de la secuencia durante las sucesivas iteraciones.

Ejemplo práctico

```
for i in range(1, 6):
    print("Número:", i)
```

Resultado:

```
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
```

Explicación: La función `range(1, 6)` genera los números desde 1 hasta 5. El número 6 no se incluye. El ciclo imprime cada número en una línea diferente.

Ejemplo: sumar los números del 1 al 5

```
suma = 0

for i in range(1, 6):
    suma = suma + i

print("La suma es:", suma)
```

Resultado:

```
La suma es: 15
```

En este ejemplo, el ciclo suma los números del 1 al 5 y guarda el resultado en la variable `suma`.

Aplicaciones del FOR

- Recorrer arreglos y listas.
- Imprimir tablas de multiplicar.
- Calcular sumas y promedios.
- Procesar registros.
- Repetir una tarea una cantidad determinada de veces.

3. Ciclo WHILE

Definición

El ciclo `while` permite repetir un bloque de instrucciones mientras una condición sea verdadera. La condición se evalúa antes de ejecutar el bloque, por lo que es posible que el ciclo no se ejecute ninguna vez si la condición es falsa desde el comienzo.

Sintaxis en Python

```
while condicion:
    instrucciones
```

Ejemplo práctico

```
contador = 1

while contador <= 5:
    print("Número:", contador)
    contador = contador + 1
```

Resultado:

```
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
```

Explicación: El ciclo comienza con el contador en 1. Mientras el contador sea menor o igual a 5, se imprime su valor y se incrementa en uno. Cuando llega a 6, la condición se vuelve falsa y el ciclo termina.

Ejemplo: solicitar una contraseña

```
contrasena = ""

while contrasena != "1234":
    contrasena = input("Ingresa la contraseña: ")

print("Contraseña correcta")
```

Explicación: El programa solicita una contraseña hasta que el usuario ingrese `1234`. Si escribe una contraseña diferente, el ciclo vuelve a ejecutarse.

Este ejemplo es únicamente demostrativo; en aplicaciones reales, las contraseñas deben manejarse con medidas de seguridad apropiadas.

Aplicaciones del WHILE

- Repetir una solicitud hasta recibir un dato válido.
- Crear menús interactivos.
- Esperar a que se cumpla una condición.
- Procesar datos hasta encontrar un valor específico.
- Ejecutar tareas cuyo número de repeticiones no se conoce de antemano.

4. Ciclo DO-WHILE

Definición

El ciclo `do-while` ejecuta primero un bloque de instrucciones y después evalúa una condición. Si la condición es verdadera, vuelve a ejecutar el bloque; si es falsa, termina.

Su característica principal es que **el bloque de instrucciones se ejecuta al menos una vez**, incluso cuando la condición es falsa desde el comienzo.